DESARROLLO WEB CON BLAZOR

Espacio de Definición Institucional

UNIDAD 2

Interfaces Web: HTML, CSS, Bootstrap y JavaScript Interop

Clases	9 a 13 (Clases 9–13 del cronograma)
Período	18 de mayo – 6 de julio de 2026
Horas	5 clases × 2 horas = 10 horas reloj
Bibliografía	Web Development with Blazor – 3rd Ed., Cap. 6, 7

Introducción a la Unidad

Esta unidad aborda la construcción visual de aplicaciones Blazor. El estudiante aprende a estructurar páginas con HTML5 semántico, aplicar estilos con CSS moderno y Bootstrap 5, utilizar el mecanismo de CSS Isolation para encapsular estilos por componente, y finalmente integrar bibliotecas JavaScript mediante el mecanismo JS Interop propio de Blazor.

Los cinco encuentros de esta unidad tienen una carga horaria más concentrada que la Unidad 1, por lo que se priorizará la práctica y el laboratorio en cada clase. Al finalizar, el estudiante habrá mejorado la interfaz del Gestor de Tareas construido en la Unidad 1.

Objetivos de Aprendizaje

- Estructurar documentos web con elementos HTML5 semánticos.
- Aplicar propiedades CSS modernas: Flexbox, Grid, variables CSS, animaciones.
- Utilizar CSS Isolation en Blazor para estilos encapsulados por componente.
- Integrar Bootstrap 5 en proyectos Blazor: grilla, componentes, utilities.
- Usar clases de Bootstrap responsive (breakpoints: sm, md, lg, xl, xxl).
- Entender cuándo y cómo usar JavaScript en una aplicación Blazor.
- Llamar funciones JavaScript desde C# con IJSRuntime y JSObjectReference.
- Invocar métodos de C# desde JavaScript (DotNetObjectReference).
- Gestionar el ciclo de vida de referencias JS para evitar memory leaks.

Clase 9 – HTML5 Semántico en Blazor

Fecha	Clase	Tema
18 de mayo de 2026	Clase 9	HTML en Blazor

Marco Teórico

HTML5 introdujo elementos semánticos que describen el significado del contenido además de su presentación. Usar semántica correcta mejora la accesibilidad (ARIA), el SEO y el mantenimiento del código. En Blazor, el HTML se escribe en la parte de marcado del archivo .razor y puede mezclar lógica C# de forma transparente.

La estructura semántica estándar de una aplicación web incluye: <header>, <nav>, <main>, <section>, <article>, <aside> y <footer>. Estos elementos reemplazan el uso indiscriminado de y permiten a lectores de pantalla y motores de búsqueda entender la jerarquía del contenido.

Conceptos Clave

- Elementos semánticos: <header>, <nav>, <main>, <section>, <article>, <aside>, <footer>
- Atributos ARIA: role, aria-label, aria-expanded, aria-live
- Jerarquía de encabezados: <h1> a <h6> y su impacto en accesibilidad
- Formularios accesibles: <label for>, <fieldset>, <legend>
- Imágenes accesibles: atributo alt y la regla de imagen decorativa (alt="")

Ejemplo de Código – Layout Semántico en Blazor

```
<!-- MainLayout.razor -->
<div class="page">
    <header class="top-row">
        <nav aria-label="Navegación principal">
            <NavLink href="/" Match="NavLinkMatch.All">Inicio</NavLink>
            <NavLink href="/tareas">Tareas</NavLink>
        </nav>
    </header>

    <main>
        <article>
            @Body
        </article>
    </main>

    <footer>
        <p>Gestor de Tareas &copy; @DateTime.Now.Year</p>
    </footer>
</div>
```

Laboratorio Guiado

1. Revisar el MainLayout.razor del proyecto y reemplazar todos los de estructura con elementos semánticos.
2. Agregar atributos aria-label a la navegación y aria-live="polite" a la zona de notificaciones.
3. Crear componente TarjetaTarea.razor usando <article> y <header> internos.
4. Validar accesibilidad básica con la extensión axe DevTools del navegador.

Ejercicios Propuestos

- Reemplazar la lista de tareas por un <table> semántico con <thead>, <tbody> y <th scope="col">.
- Agregar un <aside> en el layout que muestre las estadísticas del State Service.
- Crear un componente <SkipLink> para navegación por teclado.

Web Development with Blazor – 3rd Ed.
Capítulo 6: Styling and Layouts (primera sección)

Clase 10 – CSS Moderno y CSS Isolation

Fecha	Clase	Tema
1 de junio de 2026	Clase 10	CSS

Marco Teórico

CSS Isolation es una característica de Blazor que permite definir estilos encapsulados para un componente en un archivo .razor.css con el mismo nombre. El compilador agrega automáticamente un atributo único (data-v-xxxx) al HTML y al selector CSS, evitando colisiones entre componentes.

Además de CSS Isolation, en este encuentro se trabajan las propiedades modernas de CSS que permiten diseño adaptable: variables CSS (custom properties), Flexbox para alineación unidimensional y CSS Grid para layouts bidimensionales.

Conceptos Clave

- CSS Isolation: archivo Componente.razor.css con alcance automático
- Custom Properties: --color-primario: #1f4e79; y var(--color-primario)
- Flexbox: display:flex, flex-direction, justify-content, align-items, gap
- CSS Grid: grid-template-columns, grid-template-areas, fr unit
- Pseudoclases dinámicas en Blazor: class="@(condicion ? 'activo' : '')"

Ejemplo de Código – CSS Isolation

```
/* TareaItem.razor.css */
.tarea-card {
    border-left: 4px solid var(--color-prioridad, #888);
    padding: 0.75rem 1rem;
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 1rem;
    background: white;
    border-radius: 6px;
    box-shadow: 0 1px 3px rgba(0,0,0,.1);
    transition: transform .2s;
}
.tarea-card:hover { transform: translateY(-2px); }
.tarea-card.completada { opacity: .6; text-decoration: line-through; }
```

```
<!-- TareaItem.razor -->
<div class="tarea-card @(Tarea.Completada ? "completada" : "")"
     style="--color-prioridad: @ColorPorPrioridad(Tarea.Prioridad)">
    <span>@Tarea.Titulo</span>
</div>

@code {
    [Parameter] public TareaModel Tarea { get; set; } = default!;
    string ColorPorPrioridad(string p) => p switch {
        "Alta"  => "#dc3545",
        "Media" => "#fd7e14",
        _       => "#28a745"
    };
}
```

Laboratorio Guiado

5. Crear Tarealtem.razor.css con los estilos del bloque anterior.
6. Definir variables CSS globales en app.css: --color-primario, --color-fondo, --radio-borde.
7. Implementar modo oscuro con clase .dark-mode en <body> y sobreescritura de variables.
8. Crear grid de tarjetas con CSS Grid: 3 columnas en desktop, 1 en móvil.

Ejercicios Propuestos

- Agregar animación de entrada (keyframe fadeIn) a las tarjetas al montarse el componente.
- Crear componente Progreso.razor.css con barra de progreso animada usando CSS transitions.
- Implementar toggle de modo oscuro con checkbox y actualización de clase en <body> desde Blazor.

Web Development with Blazor – 3rd Ed.
Capítulo 6: Styling and Layouts (CSS Isolation)

Clase 11 – Bootstrap 5 en Blazor

Fecha	Clase	Tema
8 de junio de 2026	Clase 11	Bootstrap

Marco Teórico

Bootstrap 5 es el framework CSS más extendido del ecosistema web. Blazor lo incluye por defecto en las plantillas de proyecto. El sistema de grilla de Bootstrap usa 12 columnas y 6 breakpoints (xs, sm, md, lg, xl, xxl) para layouts responsive sin JavaScript adicional.

Los componentes de Bootstrap como modales, toasts, tooltips y dropdowns dependen del plugin JavaScript de Bootstrap. En Blazor se puede invocar este plugin a través de JS Interop (que se ve en la Clase 13), o usar bibliotecas wrapper como Blazorise o MudBlazor que encapsulan este comportamiento.

Conceptos Clave

- Sistema de grilla: container, row, col-{bp}-{n}
- Utilities: spacing (m-, p-), flexbox (d-flex, justify-content-), display
- Componentes: Card, Badge, Alert, Spinner, Progress, Modal (estructura)
- Forms Bootstrap: form-control, form-select, form-check, invalid-feedback
- Iconos: Bootstrap Icons (bi bi-xxx) via CDN o npm

Ejemplo de Código – Tarjeta con Bootstrap

```
<!-- TareaCard.razor -->
<div class="card mb-3 border-0 shadow-sm">
    <div class="card-header d-flex justify-content-between align-items-center
                bg-@ColorBootstrap(Tarea.Prioridad) text-white">
        <span class="fw-bold">@Tarea.Titulo</span>
        <span class="badge bg-light text-dark">@Tarea.Prioridad</span>
    </div>
    <div class="card-body">
        <p class="card-text text-muted small">
            <i class="bi bi-calendar-event"></i>
            Vence: @Tarea.FechaVencimiento?.ToString("dd/MM/yyyy")
        </p>
    </div>
    <div class="card-footer d-flex gap-2">
        <button @onclick="() => OnEditar.InvokeAsync(Tarea.Id)"
                class="btn btn-sm btn-outline-primary">
            <i class="bi bi-pencil"></i> Editar
        </button>
        <button @onclick="() => OnEliminar.InvokeAsync(Tarea.Id)"
                class="btn btn-sm btn-outline-danger">
            <i class="bi bi-trash"></i> Eliminar
        </button>
    </div>
</div>
```

Laboratorio Guiado

9. Rediseñar ListaTareas.razor usando la grilla Bootstrap: 3 columnas en lg, 1 en xs.

10. Crear barra de estadísticas con 3 Cards de Bootstrap en fila responsive.
11. Implementar modal de confirmación de eliminación (solo estructura HTML/CSS, sin JS).
12. Agregar navbar fija con Bootstrap y dropdown de filtros.

Ejercicios Propuestos

- Crear componente de paginación usando los componentes Pagination de Bootstrap.
- Agregar un Offcanvas lateral como menú en pantallas pequeñas.
- Usar utilidades Bootstrap para implementar skeleton loading mientras se espera datos.

Web Development with Blazor – 3rd Ed.
Capítulo 6: Styling and Layouts

Clase 12 – JavaScript en Aplicaciones Web Modernas

Fecha	Clase	Tema
22 de junio de 2026	Clase 12	JavaScript

Marco Teórico

Aunque Blazor permite escribir toda la lógica de UI en C#, JavaScript sigue siendo necesario para acceder a APIs del navegador que .NET no expone directamente: portapapeles, notificaciones, APIs de medios, animaciones del DOM, bibliotecas de terceros, etc.

En este encuentro se repasan fundamentos de JavaScript moderno (ES2020+) que el desarrollador Blazor necesita para escribir correctamente los módulos de interop: módulos ES, Promises, async/await, y la diferencia entre window, document y el Shadow DOM.

Conceptos Clave

- Módulos ES6: export, import, import() dinámico
- Async/Await y Promises: flujo asíncrono en JS
- Manipulación del DOM: getElementById, querySelector, classList
- APIs del navegador: localStorage, Notification, Clipboard, Fullscreen
- event.preventDefault() y event.stopPropagation() desde JS

Ejemplo de Código – Módulo JS para Interop

```
// wwwroot/js/tareas.js

export function mostrarNotificacion(titulo, mensaje) {
    if (Notification.permission === 'granted') {
        new Notification(titulo, { body: mensaje, icon: '/icon.png' });
    } else if (Notification.permission !== 'denied') {
        Notification.requestPermission().then(perm => {
            if (perm === 'granted')
                new Notification(titulo, { body: mensaje });
        });
    }
}

export function copiarAlPortapapeles(texto) {
    return navigator.clipboard.writeText(texto);
}

export function enfocarElemento(elementId) {
    const el = document.getElementById(elementId);
    el?.focus();
}
```

Laboratorio Guiado

13. Crear wwwroot/js/tareas.js con las tres funciones del ejemplo.
14. Agregar función scrollToTop() que use window.scrollTo({top:0, behavior:'smooth'}).
15. Crear función guardarEnLocalStorage(clave, json) y recuperarDeLocalStorage(clave).
16. Verificar que el módulo exporta correctamente abriendo la consola del navegador.

Ejercicios Propuestos

- Crear función printTareas() que abra el diálogo de impresión del navegador.
- Implementar función confirmarAccion(mensaje) que retorne una Promise<boolean>.
- Crear función generarIdUnico() usando crypto.randomUUID().

Web Development with Blazor – 3rd Ed.
Capítulo 7: JavaScript Interoperability (introducción)

Clase 13 – JavaScript Interoperability (JS Interop)

Fecha	Clase	Tema
29 de junio de 2026	Clase 13	JS Interop

Marco Teórico

JS Interop es el mecanismo de Blazor para la comunicación bidireccional entre C# y JavaScript. Se apoya en la interfaz IJSRuntime para llamadas desde .NET a JS, y en DotNetObjectReference para exponer métodos C# que JavaScript puede invocar.

La buena práctica es usar módulos ES6 cargados bajo demanda con IJSRuntime.InvokeAsync<IJSObjectReference>("import", ruta) en lugar de scripts globales, lo que permite code splitting y evita contaminación del namespace global.

Conceptos Clave

- IJSRuntime: InvokeAsync<T>, InvokeVoidAsync
- Módulos ES6: import() dinámico + JSObjectReference + DisposeAsync
- DotNetObjectReference.Create(this) para callbacks JS → C#
- Ciclo de vida de referencias: IAsyncDisposable
- IJSInProcessRuntime para llamadas síncronas (solo WASM)

Ejemplo de Código – Interop Completo

```
// Componente Blazor
@inject IJSRuntime JS
@implements IAsyncDisposable

@code {
    private IJSObjectReference? _modulo;

    protected override async Task OnAfterRenderAsync(bool firstRender)
    {
        if (firstRender)
            _modulo = await JS.InvokeAsync<IJSObjectReference>(
                "import", "./js/tareas.js");
    }

    async Task NotificarTareaVencida(TareaModel t)
        => await _modulo!.InvokeVoidAsync(
                "mostrarNotificacion",
                "Tarea vencida", t.Titulo);

    async Task CopiarTitulo(string titulo)
        => await _modulo!.InvokeVoidAsync("copiarAlPortapapeles", titulo);

    public async ValueTask DisposeAsync()
    {
        if (_modulo is not null)
            await _modulo.DisposeAsync();
    }
}
```

Diagrama: Flujo de JS Interop

```
Componente Blazor (.razor)
      │  await _modulo.InvokeAsync("funcion", args)
      ▼
IJSRuntime (Blazor Runtime)
      │  serializa parámetros a JSON
      ▼
Módulo JavaScript (ES6)
      │  ejecuta la función, retorna resultado
      ▼
IJSRuntime deserializa respuesta → Task<T>
      │
      ▼
Componente Blazor recibe resultado tipado
```

Laboratorio Guiado

17. Inyectar IJSRuntime en el componente principal e importar tareas.js en OnAfterRenderAsync.
18. Agregar botón 'Copiar enlace' que invoque copiarAlPortapapeles con la URL de la tarea.
19. Mostrar notificación del navegador cuando una tarea tenga menos de 24 hs para vencer.
20. Implementar IAsyncDisposable y liberar el módulo correctamente.

Ejercicios Propuestos

- Crear función JS que retorne la resolución de pantalla actual; mostrarla en un tooltip.
- Implementar callback C# desde JS: cuando el usuario sale de la pestaña (visibilitychange), pausar un timer en C#.
- Usar window.matchMedia para detectar preferencia de modo oscuro del sistema operativo.

Web Development with Blazor – 3rd Ed.
Capítulo 7: JavaScript Interoperability

Mini Proyecto de la Unidad – Interfaz Visual del Gestor

Aplicar todos los conceptos de la Unidad 2 para rediseñar la interfaz visual del Gestor de Tareas construido en la Unidad 1.

REQUERIMIENTOS DE INTERFAZ

HTML5 semántico: header, nav, main, article, footer en todos los layouts
CSS Isolation: cada componente con su archivo .razor.css
Variables CSS globales: paleta de colores y tipografía en app.css
Bootstrap 5: grilla responsive, cards, badges, botones con iconos
Modo oscuro: toggle visible con persistencia en localStorage vía JS Interop
JS Interop: notificaciones de tareas vencidas, copiar al portapapeles
Modal de confirmación para eliminar (HTML+CSS, sin JS externo)
Diseño mobile-first: completamente usable en pantallas de 375px

Criterios de Evaluación

- Semántica HTML5 aplicada correctamente (sin divitis).
- CSS Isolation implementado en al menos 3 componentes.
- Grilla Bootstrap responsive verificada en mobile y desktop.
- JS Interop con módulo ES6 y manejo correcto de DisposeAsync.
- Modo oscuro funcional con persistencia entre sesiones.

Bibliografía de la Unidad

Capítulo	Título	Clases
Cap. 6	Styling and Layouts	9–11
Cap. 7	JavaScript Interoperability	12–13